

AGENTIC AI CAPSTONE PROGRAM

Day 1 Lab Guide Configurable Rule Engine

Automated Utility & Telecom Billing State Machine with LangGraph — Priority Rule Matching, Smart Matching, Adjustment Coding, Tolerance Write-Off, and Exception Routing

VM ONLY

Azure AI Foundry

VS Code

LangGraph

Notebook file: 1.ipynb

Scenario: Automated Customer Payment & Billing Reconciliation System (GreenGrid Utilities)

Format: Self-contained lab — environment setup, business context, and a full cell-by-cell code walkthrough

Environment: All work is performed on the assigned training Virtual Machine only

Contents

1. Environment Setup (VM, Azure AI Foundry, VS Code)

2. The Business Scenario Behind This Notebook

3. Cell-by-Cell Walkthrough of 1.ipynb (32 cells)

HOW TO USE THIS LAB GUIDE

Work through Section 1 first to get your Virtual Machine, Azure AI Foundry project, model deployment, and VS Code environment ready. Section 2 gives you the business context behind the code. Section 3 walks through the notebook cell by cell — run each cell in VS Code on your VM in the order shown, read the explanation before you run it, and compare your output with the expected output shown under each cell.

Section 1 — Environment Setup (Perform Every Step on the VM)

DO THIS ON THE VIRTUAL MACHINE ONLY

Every action in this section — opening a browser, running a command, writing code — must be carried out inside the training Virtual Machine that was assigned to you. Do not install Python packages or keep any project files on your personal laptop or desktop. If you close the VM session, nothing you did outside it will be there when you reconnect, and code run on your personal machine will not have access to the network access or grading environment used for this course.

1.1 Open and Connect to Your Virtual Machine

Step 1 — Connect to the VM

1. Open the remote desktop / lab-access tool your training provider gave you (for example, Remote Desktop Connection for a Windows VM, or the RDP/SSH link from your lab portal).
2. Enter the VM's IP address / hostname and the username and password supplied to you for this course.
3. Wait for the desktop to load fully before continuing — this confirms the VM is running and reachable.

1.2 Create Your Azure AI Foundry Resource and Project

Step 2 — Create the Foundry project

1. In the browser inside the VM, go to <https://ai.azure.com> (the Microsoft Foundry portal) and sign in with the Azure account provided for this training.
2. On the landing page select **Create project**.
3. Give the project a name, for example `greengrid-reconciliation-proj`.
4. When prompted for a hub, select **Create new hub** (a hub is the shared container that holds the underlying Foundry resource, storage, and connections). Name it, for example `greengrid-ops-hub`.
5. Review the resources the wizard will create, then select **Create**. Provisioning usually takes one to three minutes.
6. Once the project opens, note the project endpoint shown on the project Overview page — you will paste this into the notebook shortly.

1.3 Deploy the gpt-5 Model

Step 3 — Deploy the model from the catalog

1. Inside your Foundry project, open **Model catalog** from the left navigation.
2. Search for `gpt-5` and open its model card.
3. Select **Deploy** (or **Deploy base model** if you are in the classic Foundry view under Models + endpoints).
4. Give the deployment a name — keep it simple, for example `gpt-5`, since this is the exact string the notebook uses for `MODEL_DEPLOYMENT_NAME`.
5. Leave Deployment type on the default suggested option (typically Global Standard) and select **Deploy / Confirm**.

IF DEPLOYMENT FAILS WITH A QUOTA ERROR

If you see a banner such as "Quota exceeded", "Insufficient quota", or the deployment status gets stuck because there is no available Tokens-Per-Minute (TPM) quota for gpt-5 under Standard or Global Standard in your region, do the following:

1. Go back to the deployment screen and open the Deployment type dropdown.
2. Change it from Global Standard (or Standard) to **Global Batch**.
3. Select Deploy again. Global Batch deployments are evaluated against a separate, usually more available, quota pool, so this frequently unblocks a deployment when the interactive tiers are exhausted.
4. You can confirm your current quota at any time under Management → Quota in the left navigation of the Foundry portal.

Note: Global Batch deployments process requests asynchronously (submit-and-poll) rather than returning an answer instantly on the same request. It is a good fallback to get a working deployment when interactive quota is unavailable during a training session; for the fastest request/response cycle inside the notebook prefer Standard or Global Standard once you are able to secure quota for one of those tiers.

1.4 Set Your Credentials & Initialize a Reusable Client

Step 4 — Set your credentials as environment variables and build one shared client

1. In the Foundry project, open **Models + endpoints** (or the deployment you just created) and copy the Target URI / endpoint value.
2. Copy one of the Keys shown alongside it.
3. Open **1.ipynb** in VS Code. The first two cells of the notebook (Cells 1 and 2) already do the credential and client setup for you — just fill in your own endpoint and key in Cell 1 before you run it:

Python — Cell 1 of the notebook

```
os.environ["AZURE_OPENAI_API_KEY"] = "<AZURE_OPENAI_API_KEY>"
os.environ["AZURE_OPENAI_ENDPOINT"] = "<AZURE_OPENAI_ENDPOINT>"
```

Cell 2 then builds one shared `client` object (an `OpenAIChatClient` from the Microsoft Agent Framework, wrapping an `AsyncAzureOpenAI` instance) from those two environment variables. Run Cells 1 and 2 first, every time you open the notebook, before any other cell. Every later cell or notebook that needs to talk to your model deployment can reuse this same `client` variable instead of constructing a brand-new client of its own.

TREAT THIS CELL LIKE A SECRET

Keeping credentials as environment variables set inside the notebook is simple and fine for this training environment, but it does mean the key is now sitting in a file. Don't commit this notebook to a shared or public Git repository once you've filled in your real key, and don't share the file with the key still in it. None of the notebooks in this capstone ship with a real key baked in — every credential placeholder you see below is exactly that, a placeholder, and it is your responsibility to fill in your own value and keep it out of anything you hand in or share.

1.5 Open VS Code

USE VS CODE ON THE VM — NOT YOUR PERSONAL COMPUTER

Do all of your work in the VS Code instance already installed on the Virtual Machine. Do not write, edit, or execute any of this lab's code in a VS Code instance on your personal laptop, even if it feels more convenient. The VM has the network access and pre-approved configuration this course depends on.

Step 5 — Open VS Code

VS Code is already installed on your training VM — there is nothing to download or install. Open it directly from the desktop shortcut, Start menu (Windows), or Applications/taskbar (Linux).

Once it's open, quickly check the Extensions panel (Ctrl+Shift+X) and confirm these two extensions are already enabled:

- **Python** (Microsoft) — language support, environment selection, debugging
- **Jupyter** (Microsoft) — lets you open and run .ipynb files directly in VS Code

If either extension is somehow missing, install just that extension from the Extensions panel — there's no need to reinstall VS Code itself.

1.6 Set Up the Python Environment on the VM

Step 6 — Create a virtual environment and install packages

Open the integrated terminal in VS Code (Ctrl+`) inside your project folder on the VM:

Windows VM (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install azure-identity azure-ai-projects azure-core openai langgraph
pip install uv
uv pip install agent-framework
```

Linux VM (bash)

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install azure-identity azure-ai-projects azure-core openai langgraph
pip install uv
uv pip install agent-framework
```

In VS Code, open the Command Palette (Ctrl+Shift+P) → **Python: Select Interpreter** → choose the `.venv` environment you just created, so the notebook kernel uses these packages.

WHY UV FOR AGENT-FRAMEWORK

uv is a fast, drop-in-compatible replacement for pip. Once it's installed, `uv pip install <package>` installs into your already-activated `.venv` exactly like pip would, just considerably faster — useful here since **agent-framework** pulls in a fair number of sub-packages. The rest of your packages can stay on plain pip; there's no need to migrate anything else.

1.7 Open the Notebook in VS Code

Step 7 — Open and select the kernel

1. In VS Code on the VM, use **File** → **Open Folder** and open the folder that contains 1.ipynb.
2. Click **1.ipynb** in the Explorer panel to open it in the Notebook editor.
3. In the top-right of the notebook, select **Select Kernel** → **Python Environments** and choose the **.venv** interpreter you created in Step 6.
4. Run cells top to bottom with Shift+Enter, following the walkthrough in Section 3 of this guide.

Section 2 — The Business Scenario Behind This Notebook

Every notebook in this three-day capstone is built around one running scenario: an automated system for reconciling customer payments against open bills for **GreenGrid Utilities**, a fictional electric & water utility provider. Understanding this scenario makes the code below much easier to follow, because every function, state field, and test case in the notebooks maps directly back to a piece of this business problem.

2.1 The Problem

Utility and telecom companies receive customer payments through many channels — bank transfer, online bill-pay, mobile wallet — and must apply each payment to the correct open bill accurately and quickly. In practice, a customer's payment rarely lines up perfectly with what was billed. Customers pay lump sums covering several billing periods, short-pay because a meter reading looked wrong, reference an old account number instead of the current bill, or send a payment with no identifying reference at all. Reviewing every one of these mismatches by hand does not scale, so this capstone automates the matching, adjustment detection, and settlement decision using an agentic AI workflow built with LangGraph and Microsoft Foundry.

2.2 The Three Source Documents Being Reconciled

Source	What it represents
Payment Transaction	The incoming money notification from the bank, payment gateway, or mobile wallet: account reference, amount, and channel.
Billing AR Record	The utility's own billing system record of what is actually owed — the open bill amount, meter ID, and bill date that should apply.
Dispute / Adjustment Note	Emails, IVR call transcripts, or field-technician notes explaining why a payment might be short (e.g., a disputed meter reading or an outage credit).

In the notebook, these three sources show up as `payment_transaction` , `billing_ar` , and `dispute_note` — the raw inputs carried inside every workflow state object you will see below.

2.3 The Six Possible End States

Every payment that enters the system is ultimately resolved into one of six end states:

#	End State	Meaning
1	OPEN	Payment ingested, but no matching open bill or dispute note has been found yet.
2	PARTIAL MATCH	A match was found, but the amount paid is short of what was billed.
3	CLOSED	Payment and open bill fully agree (or the gap is within tolerance) — fully settled.
4	UAC (Un-applied Cash)	The customer is recognized, but no open bill/account reference could be matched.
5	UIC (Un-Identified Cash)	The payment is missing the identifiers needed to match it to anything.

6	QUERY	The case is ambiguous, contains a possible threat, or is too low-confidence to resolve automatically — a human reviewer is needed.
---	-------	--

You will see these exact state names (OPEN, PARTIAL MATCH, CLOSED, UAC, UIC, QUERY) assigned to `current_state` throughout the notebook code.

2.4 Adjustment Reason Codes

When a mismatch is found, the system tags it with a standard reason code so it can be tracked and reported on:

Code	Category	Typical Trigger
U01	Meter Reading Dispute	Customer disputes an estimated or unusually high meter reading.
U02	Service Outage Credit	A credit is issued for an extended unplanned service outage.
U03	Estimated Bill Correction	An actual meter read replaces a prior system-estimated bill.
U04	Assistance / Discount Adjustment	A senior-citizen, low-income, or subsidy program discount is applied.
U05	Late Fee Waiver	A late-payment penalty is waived as a goodwill or first-occurrence gesture.

2.5 How the Three Days Build on Each Other

The three notebooks are a progressive build, not three unrelated exercises:

- **Day 1 — Configurable Rule Engine.** Deterministic priority-rule matching (2-way / 3-way / fuzzy), an AI-style Smart Matching scorer, adjustment coding, tolerance-based auto write-off, and exception routing, wired together as a LangGraph `StateGraph`.
- **Day 2 — Build & Validate.** Adds Supervisor-pattern multi-agent orchestration, an MCP tool server, an A2A message-contract hand-off, a Voice Agent intake lab, a Chroma vector store holding unstructured dispute/outage notes, a Foundry-grounded reasoning node, a regression test suite, a deployment manifest, and observability metrics.
- **Day 3 — Deliver & Govern.** Adds an anti-prompt-injection scanner and a PII masking guardrail around the Day 1 + Day 2 pipeline, a real-time audit-streaming logger, Partial Payments allocation, a D365 ERP journal-entry integration, batch reporting, an agent maturity scorecard, a risk register, a RAID log, and a secured pipeline stress-tested against a clean case, an injection attack, and a sensitive-data-exposure case.

WHY THIS CONTEXT MATTERS WHILE YOU CODE

Whenever a cell mentions "Meridian Manufacturing", "Coastal Retail Group", "Sunrise Apartments", or a specific bill like BILL-4471, it is exercising one of the scenario examples above (a perfect match, a meter-reading dispute, a tolerance write-off, or an unmatched-cash case). Recognizing which example you are looking at makes it much easier to predict what the output of a cell should be before you run it.

Section 3 — Cell-by-Cell Walkthrough of 1.ipynb

This notebook has 32 code cells. Run them in order, top to bottom, in VS Code on your VM. Each card below shows the cell number, a plain-language explanation of what it does and why, the exact code, and the output you should expect to see when you run it yourself.

Cell 1 — Set your Azure OpenAI credentials as environment variables

This is the very first cell of the notebook and the only one you need to edit before running anything. It sets `AZURE_OPENAI_API_KEY` and `AZURE_OPENAI_ENDPOINT` as process environment variables using `os.environ[...]`, using the exact variable names the `openai` SDK's Azure client looks for automatically. Replace the two placeholder strings with your own endpoint and key from Section 1.4 before you run this cell — every later cell (and every later notebook) that talks to your model deployment reads its credentials from these same two environment variables instead of redefining them.

CODE

```
import os

# Set / Export environment variables
os.environ["AZURE_OPENAI_API_KEY"] = "<AZURE_OPENAI_API_KEY>"
os.environ["AZURE_OPENAI_ENDPOINT"] = "<AZURE_OPENAI_ENDPOINT>"

# Access / Read environment variables
api_key = os.environ.get("AZURE_OPENAI_API_KEY")
print("API Key set:", api_key)
```

OUTPUT

```
API Key set: <AZURE_OPENAI_API_KEY>
```

Cell 2 — Build one reusable OpenAIChatClient

Rather than constructing a brand-new client every time a cell needs to call the model, this cell builds exactly one shared `client` object, up front, that every later cell and notebook can reuse. It first creates an `AsyncAzureOpenAI` instance from the `openai` SDK — which automatically reads the two environment variables from the previous cell, so no endpoint or key is passed in directly — and then wraps it in a Microsoft Agent Framework `OpenAIChatClient`, pointed at your `gpt-5` deployment. The whole block is wrapped in try/except, matching the defensive style used throughout this capstone: if `agent-framework` isn't installed yet, or your credentials aren't valid, you get a clear one-line message here instead of a crash that blocks every other cell in the notebook.

CODE

```
try:
    from openai import AsyncAzureOpenAI
    from agent_framework import Agent
    from agent_framework.openai import OpenAIChatClient

    # 1. Initialize AsyncAzureOpenAI with your API key
    azure_client = AsyncAzureOpenAI(api_version="2024-06-01")

    # 2. Pass the client to OpenAIChatClient
    client = OpenAIChatClient(
        async_client=azure_client,
        model="gpt-5", # Deployment name in Azure
    )
    print("Reusable OpenAIChatClient initialized successfully as `client`.")
except Exception as err:
    print(f"Failed to initialize the reusable Agent Framework client: {err}")
```

OUTPUT

```
Reusable OpenAIChatClient initialized successfully as `client`.
```

Cell 3 — Reset the telemetry log file

This is a housekeeping cell. It checks whether a `system.log` file already exists in the working folder from a previous run, and if so, empties it (overwrites it with an empty string). Running this first guarantees that every log entry you see later in this lab was produced by this run of the notebook, not left over from a previous session.

CODE

```
from pathlib import Path

# Clear system.log if it exists (fresh run for this session)
log_file = Path("system.log")
if log_file.exists():
    log_file.write_text("", encoding="utf-8")
```

This cell produces no printed output.

Cell 4 — Define the JSONStateDeltaLogger class (constructor)

This defines the constructor (`__init__`) of a small custom logger class, `JSONStateDeltaLogger` , that you will reuse throughout the entire three-day capstone. It wraps Python's standard `logging` module rather than replacing it, and attaches two handlers: a `FileHandler` that appends every log line to `system.log` , and a `StreamHandler` that also prints the same line to the notebook output, so you can watch events happen live and still have a permanent audit trail on disk. It uses a bare `"%(message)s"` formatter, meaning whatever string you pass in is written out exactly as-is — this matters because the next cell will pass in a JSON string, so each log line becomes one valid, independently-parseable JSON object.

CODE

```
import json
import logging
import sys
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, Optional

class JSONStateDeltaLogger:
    def __init__(self, log_file_path: str = "system.log") -> None:
        self.log_file = Path(log_file_path)
        self.logger = logging.getLogger("GreenGridReconciliationTelemetry")
        self.logger.setLevel(logging.INFO)
        self.logger.handlers.clear()

        file_handler = logging.FileHandler(self.log_file, mode="a", encoding="utf-8")
        file_formatter = logging.Formatter("%(message)s")
        file_handler.setFormatter(file_formatter)
        self.logger.addHandler(file_handler)

        console_handler = logging.StreamHandler(sys.stdout)
        console_handler.setFormatter(file_formatter)
        self.logger.addHandler(console_handler)
```

This cell produces no printed output.

Cell 5 — Add the `log_state_delta` method to the logger

This cell defines a standalone function, `log_state_delta`, and then attaches it onto the `JSONStateDeltaLogger` class with `JSONStateDeltaLogger.log_state_delta = log_state_delta`. This is a monkey-patching pattern — it lets the notebook build up a class's behavior incrementally across several cells, which is convenient for teaching but is not something you'd normally do in production code (there, this method would simply live inside the class body). The method itself builds a structured `event_record` dictionary capturing the transaction ID, account number, the state transition (from → to), which priority rule fired, any adjustment code, and a free-form `delta_payload` dictionary for extra details. It serializes that record to a JSON string and writes it through the logger. If serialization fails for any reason, the exception is caught and reported to `stderr` instead of crashing the notebook.

CODE

```
def log_state_delta(
    self,
    transaction_id: str,
    account_number: Optional[str],
    previous_state: str,
    new_state: str,
    priority_rule_applied: Optional[int],
    delta_payload: Dict[str, Any],
    adjustment_code: Optional[str] = None,
) -> Dict[str, Any]:
    event_record = {
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "telemetry_event": "STATE_DELTA_TRANSITION",
        "transaction_id": transaction_id,
        "account_number": account_number,
        "state_transition": {
            "from": previous_state,
            "to": new_state,
        },
        "rule_context": {
            "priority_rule": priority_rule_applied,
            "adjustment_code": adjustment_code,
        },
        "delta_payload": delta_payload,
    }

    try:
        log_line = json.dumps(event_record, ensure_ascii=False)
        self.logger.info(log_line)
    except Exception as ex:
        sys.stderr.write(f"Failed to stream telemetry delta log: {str(ex)}\n")

    return event_record

JSONStateDeltaLogger.log_state_delta = log_state_delta
```

This cell produces no printed output.

Cell 6 — Test the logger with a perfect-match example

This is the first live test of the logger. It instantiates `JSONStateDeltaLogger("system.log")` and logs a transition for transaction TXN-MER-6120 moving from OPEN to CLOSED — a perfect 2-way match between Meridian Manufacturing's payment and bill BILL-4471, so there is no adjustment code and Priority Rule 4 is recorded as the match.

CODE

```
try:
    logger = JSONStateDeltaLogger("system.log")

    test_result = logger.log_state_delta(
        transaction_id="TXN-MER-6120",
        account_number="ACC-77210",
        previous_state="OPEN",
        new_state="CLOSED",
        priority_rule_applied=4,
        adjustment_code=None,
        delta_payload={
            "customer_name": "Meridian Manufacturing",
            "bill_id": "BILL-4471",
            "amount_paid": 482.50,
            "match_type": "2-Way",
            "status_reason": "Perfect 2-Way Match Successful",
        },
    )
    print("\nTelemetry event logged successfully.")
except Exception as err:
    print(f"Error during logger execution: {err}")
```

OUTPUT

```
{"timestamp": "2026-08-16T12:56:29.330246+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "TXN-MER-6120", "account_number": "ACC-77210", "state_transition": {"from":
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 4, "adjustment_code": null},
"delta_payload": {"customer_name": "Meridian Manufacturing", "bill_id": "BILL-4471",
"amount_paid": 482.5, "match_type": "2-Way", "status_reason": "Perfect 2-Way Match
Successful"}}

Telemetry event logged successfully.
```

Cell 7 — Read back the contents of system.log

A verification cell: it opens `system.log` from disk and prints its full contents. You should see exactly one JSON line so far — the event written by the previous cell. This is a useful pattern to reuse any time you want to double check that telemetry is really landing on disk and not just being printed to the console.

CODE

```
try:
    log_path = Path("system.log")
    if log_path.exists():
        content = log_path.read_text(encoding="utf-8")
        print("--- System Log File Contents ---")
        print(content)
    else:
        print("system.log file not found.")
except Exception as err:
    print(f"Failed to read system.log: {err}")
```

OUTPUT

```
--- System Log File Contents ---
{"timestamp": "2026-08-16T12:56:29.330246+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "TXN-MER-6120", "account_number": "ACC-77210", "state_transition": {"from":
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 4, "adjustment_code": null},
"delta_payload": {"customer_name": "Meridian Manufacturing", "bill_id": "BILL-4471",
"amount_paid": 482.5, "match_type": "2-Way", "status_reason": "Perfect 2-Way Match
Successful"}}
```

Cell 8 — A1: Agent Architecture Patterns — typing imports

This comment banner marks the start of the A1 module (Agent Architecture Patterns) and imports the typing helpers — `TypedDict`, `Dict`, `List`, `Optional`, and `Any` — used to formalize the shape of every data structure that flows through the graph. `TypedDict` lets you describe the exact shape of a dictionary while still using it as a plain Python dict at runtime — you get IDE autocomplete and static-analysis checking without any runtime overhead.

CODE

```
# =====
# A1: Agent Architecture Patterns – Core Data Schemas
# Typed payloads represent the 3 core data inputs for reconciliation:
# Payment Transaction, Billing AR record, and Dispute/Adjustment Note.
# =====
from typing import Dict, List, Optional, Any, TypedDict, Literal
```

This cell produces no printed output.

Cell 9 — Define the three source-data schemas

This defines three `TypedDict` classes that formalize the three source documents from Section 2.2:

`PaymentTransactionPayload` (account, amount, channel), `BillingARPayload` (bill ID, meter ID, total amount), and `DisputeNotePayload` (dispute ID and the free-text notes). These three schemas are the strongly-typed version of the three inputs you read about above.

CODE

```
class PaymentTransactionPayload(TypedDict):
    transaction_id: str
    account_number: Optional[str]
    customer_name: Optional[str]
    amount_paid: float
    payment_date: str
    reference_number: Optional[str]
    payment_channel: str

class BillingARPayload(TypedDict):
    bill_id: str
    account_number: str
    customer_name: str
    meter_id: Optional[str]
    bill_date: str
    total_amount: float
    is_open: bool

class DisputeNotePayload(TypedDict):
    dispute_id: str
    account_number: Optional[str]
    unstructured_notes: str
    source_channel: str
```

This cell produces no printed output.

Cell 10 — Define the UtilityReconciliationState schema

UtilityReconciliationState is the single object that flows through every node of the LangGraph workflow. It groups four kinds of information: the three raw payloads; state-machine controls (**current_state** , **priority_rule_matched** , **match_type**); the Smart Matching attributes (**smart_match_candidates** , **smart_match_confidence**); the adjustment & settlement ledger (**adjustment_code** , **approved_settlement_amount** , **held_variance_amount** , **auto_write_off_amount**); and audit / human-in-the-loop fields. Every node function you build later reads from and returns a dictionary shaped like this.

CODE

```
class UtilityReconciliationState(TypedDict):
    # Raw Input Master Data Sources (Data Ingestion use case)
    payment_transaction: PaymentTransactionPayload
    billing_ar: Optional[BillingARPayload]
    dispute_note: Optional[DisputeNotePayload]

    # State Machine Controls
    current_state: str
    priority_rule_matched: Optional[int]
    match_type: Optional[str]

    # Smart Matching (AI-suggested) Attributes
    smart_match_candidates: List[Dict[str, Any]]
    smart_match_confidence: float

    # Adjustment & Financial Settlement Ledger
    adjustment_code: Optional[str]
    adjustment_category: Optional[str]
    approved_settlement_amount: float
    held_variance_amount: float
    auto_write_off_amount: float

    # Audit & HITL Attributes
    audit_notes: List[str]
    requires_human_review: bool
    ai_confidence_score: float
```

This cell produces no printed output.

Cell 11 — Write the `create_initial_reconciliation_state` factory function

A small factory function that takes a payment payload (required) plus an optional billing record and dispute note, and returns a brand-new `UtilityReconciliationState` dictionary with every field defaulted sensibly: `current_state` starts at "OPEN", all money fields start at 0.0, and `audit_notes` starts with one entry ("State initialized to OPEN."). Using a factory function like this avoids repeating this boilerplate every time you need to start a new case.

CODE

```
def create_initial_reconciliation_state(
    payment_payload: PaymentTransactionPayload,
    billing_payload: Optional[BillingARPayload] = None,
    dispute_payload: Optional[DisputeNotePayload] = None,
) -> UtilityReconciliationState:
    return {
        "payment_transaction": payment_payload,
        "billing_ar": billing_payload,
        "dispute_note": dispute_payload,
        "current_state": "OPEN",
        "priority_rule_matched": None,
        "match_type": None,
        "smart_match_candidates": [],
        "smart_match_confidence": 0.0,
        "adjustment_code": None,
        "adjustment_category": None,
        "approved_settlement_amount": 0.0,
        "held_variance_amount": 0.0,
        "auto_write_off_amount": 0.0,
        "audit_notes": ["State initialized to OPEN."],
        "requires_human_review": False,
        "ai_confidence_score": 1.0,
    }
```

This cell produces no printed output.

This defines `data_ingestion_node`, the first real business-logic node and an implementation of the **Data Ingestion** use case discussed on Day 1: reading a payment record from an email/ERP export and validating that the required fields (`transaction_id` , `amount_paid`) are actually present before the record is allowed further into the pipeline. If a required field is missing, the function flags `requires_human_review` rather than letting a malformed record silently proceed.

CODE

```
# =====
# Use Case: Data Ingestion
# Reads payment/billing records from email or ERP export files, validates
# required fields are present, and loads them into the pipeline.
# =====
def data_ingestion_node(state: Dict[str, Any]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))

    payment = state.get("payment_transaction", {})
    required_fields = ["transaction_id", "amount_paid"]
    missing = [f for f in required_fields if not payment.get(f)]

    if missing:
        audit_notes.append(f>Data Ingestion validation failed. Missing fields: {missing}.)
        updated_state["requires_human_review"] = True
    else:
        audit_notes.append(
            f>Data Ingestion validated payment {payment.get('transaction_id')} "
            f"from channel '{payment.get('payment_channel', 'unknown')}'."
        )

    updated_state["audit_notes"] = audit_notes
    return updated_state
```

This cell produces no printed output.

Cell 13 — Test Data Ingestion with a sample payment

Builds one sample payment (Meridian Manufacturing, \$560.00, referencing BILL-4471) and runs it through `data_ingestion_node` directly, outside of any graph, so you can see the validation pass and read the audit note it appends.

CODE

```
sample_payment_ingest: PaymentTransactionPayload = {
    "transaction_id": "TXN-6055",
    "account_number": "ACC-77210",
    "customer_name": "Meridian Manufacturing",
    "amount_paid": 560.00,
    "payment_date": "2026-08-10",
    "reference_number": "BILL-4471",
    "payment_channel": "ACH_BANK_TRANSFER",
}

try:
    ingest_state = data_ingestion_node({"payment_transaction": sample_payment_ingest,
    "audit_notes": []})
    print("Data Ingestion Result:")
    for note in ingest_state["audit_notes"]:
        print(f" - {note}")
except Exception as err:
    print(f"Data ingestion test failed: {err}")
```

OUTPUT

```
Data Ingestion Result:
- Data Ingestion validated payment TXN-6055 from channel 'ACH_BANK_TRANSFER'.
```

This defines the **Remittance Processing** use case: `remittance_processing_node` extracts structured fields out of the unstructured dispute text using lightweight rule-based parsing — a regex pulls out a claimed dollar amount, and a keyword dictionary (`ADJUSTMENT_KEYWORDS`) maps phrases like "meter read" or "outage" to a candidate adjustment code (U01–U05). This is a deliberately lightweight, deterministic first pass; Notebook 2 upgrades this same job to real vector search plus a Foundry-grounded LLM call.

CODE

```
# =====
# Use Case: Remittance Processing
# Extracts structured remittance/dispute fields (claimed short-pay amount,
# keyword-based reason hint) from unstructured dispute text using
# lightweight rule-based parsing. (Deep NLP/RAG extraction is built in
# Notebook 2 using vector search + grounded LLM reasoning.)
# =====
import re

ADJUSTMENT_KEYWORDS = {
    "U01": ["meter read", "estimated high", "meter reading", "re-read"],
    "U02": ["outage", "sla credit", "power outage", "service interruption"],
    "U03": ["estimated bill", "bill correction", "actual read"],
    "U04": ["subsidy", "discount", "senior citizen", "low-income", "assistance program"],
    "U05": ["late fee", "penalty waiver", "goodwill"],
}

def remittance_processing_node(state: Dict[str, Any]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    dispute = state.get("dispute_note")

    if not dispute:
        audit_notes.append("Remittance Processing: no dispute/adjustment note attached.")
        updated_state["audit_notes"] = audit_notes
        return updated_state

    notes_text = dispute.get("unstructured_notes", "").lower()
    amount_match = re.search(r"\$(\d+(?:\.\d{1,2})?)", notes_text)
    claimed_amount = float(amount_match.group(1)) if amount_match else None

    matched_code = None
    for code, keywords in ADJUSTMENT_KEYWORDS.items():
        if any(kw in notes_text for kw in keywords):
            matched_code = code
            break

    updated_state["adjustment_code"] = matched_code
    audit_notes.append(
        f"Remittance Processing extracted claimed_amount={claimed_amount} "
        f"keyword_hint_code={matched_code} from dispute note {dispute.get('dispute_id')}."
    )
    updated_state["audit_notes"] = audit_notes
    return updated_state
```

This cell produces no printed output.

Cell 15 — Test Remittance Processing with a sample dispute note

Runs a sample dispute note ("Meter read appears estimated high in error (\$50 credit requested)") through the node and prints which adjustment code the keyword heuristic assigned. You should see code **U01** (Meter Reading Dispute).

CODE

```
sample_dispute: DisputeNotePayload = {
    "dispute_id": "DSP-330",
    "account_number": "ACC-77210",
    "unstructured_notes": "Meter read appears estimated high in error ($50 credit requested).",
    "source_channel": "EMAIL",
}

try:
    remit_state = remittance_processing_node({"dispute_note": sample_dispute, "audit_notes":
[]})
    print("Remittance Processing Result:")
    for note in remit_state["audit_notes"]:
        print(f" - {note}")
    print(f"Adjustment Code Hint: {remit_state['adjustment_code']}")
except Exception as err:
    print(f"Remittance processing test failed: {err}")
```

OUTPUT

```
Remittance Processing Result:
 - Remittance Processing extracted claimed_amount=50.0 keyword_hint_code=U01 from dispute note
DSP-330.
Adjustment Code Hint: U01
```

This is the heart of the Day 1 rule engine and implements the **Payment Matching** use case (rule-based 2-way / 3-way / fuzzy matching). `fuzzy_similarity` uses Python's `difflib.SequenceMatcher` to score how close two text strings are, for the case where an exact account-number match fails but the customer name is clearly the same. `execute_priority_rules_node` then checks Priority Rules 1 through 6 in order — account+bill+amount, account+amount only, a 3-way check reinforced by a dispute note, and finally the fuzzy-name fallback — stopping at the first rule that matches and recording exactly which one fired.

CODE

```
# =====
# Use Case: Payment Matching
# Sequential rule-based 2-way and 3-way priority matching (Rules 1-6),
# with a fuzzy-matching fallback for near-miss account/customer name text.
# =====
from difflib import SequenceMatcher

def fuzzy_similarity(a: str, b: str) -> float:
    if not a or not b:
        return 0.0
    return SequenceMatcher(None, a.strip().lower(), b.strip().lower()).ratio()

def execute_priority_rules_node(state: Dict[str, Any]) -> Dict[str, Any]:
    payment = state.get("payment_transaction", {})
    billing = state.get("billing_ar")
    dispute = state.get("dispute_note")

    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))

    if not billing:
        updated_state["priority_rule_matched"] = None
        updated_state["match_type"] = None
        audit_notes.append("No Billing AR record available; no rule can be evaluated.")
        updated_state["audit_notes"] = audit_notes
        return updated_state

    same_account = payment.get("account_number") == billing.get("account_number")
    same_bill_ref = payment.get("reference_number") == billing.get("bill_id")
    same_amount = payment.get("amount_paid") == billing.get("total_amount")

    # Priority 1: Account Number + Bill Number + Amount
    if same_account and same_bill_ref and same_amount:
        updated_state["priority_rule_matched"] = 1
        updated_state["match_type"] = "2-Way"
        audit_notes.append("Priority Rule 1 Matched: Account + Bill Number + Amount.")
    # Priority 3: Account + Bill Number + Bill Date + Amount
    elif same_account and same_bill_ref and dispute is None:
        updated_state["priority_rule_matched"] = 3
        updated_state["match_type"] = "2-Way"
        audit_notes.append("Priority Rule 3 Matched: Account + Bill Number + Bill Date + Amount.")
    # Priority 4: Account Number + Amount only (most common)
    elif same_account and same_amount:
        updated_state["priority_rule_matched"] = 4
        updated_state["match_type"] = "2-Way"
        audit_notes.append("Priority Rule 4 Matched: Account + Amount (standard bill-pay reference).")
    # Priority 6 (3-Way, with dispute note reinforcing the match)
    elif same_account and dispute is not None:
        updated_state["priority_rule_matched"] = 6
        updated_state["match_type"] = "3-Way"
        audit_notes.append("Priority Rule 6 Matched: Account + Bill Number + Amount, reinforced
```

```

by dispute note (3-Way).")
    else:
        # Fuzzy fallback: try matching on customer name text similarity
        name_similarity = fuzzy_similarity(payment.get("customer_name", ""),
billing.get("customer_name", ""))
        if name_similarity >= 0.82 and same_amount:
            updated_state["priority_rule_matched"] = 2
            updated_state["match_type"] = "2-Way (Fuzzy)"
            audit_notes.append(f"Fuzzy Match Matched (Priority 2 variant): customer name
similarity={name_similarity:.2f}.")
        else:
            updated_state["priority_rule_matched"] = None
            updated_state["match_type"] = None
            audit_notes.append(f"No exact or fuzzy match found across Priority Rules 1-6 (best
name similarity={name_similarity:.2f}).")

    updated_state["audit_notes"] = audit_notes
    return updated_state

```

This cell produces no printed output.

Builds Example A from the business scenario: Meridian Manufacturing pays exactly \$482.50 against bill BILL-4471, which is also exactly \$482.50. Running this through `execute_priority_rules_node` directly (no graph yet) should report Priority Rule 1 and match type "2-Way".

CODE

```
sample_billing_a: BillingARPayload = {
    "bill_id": "BILL-4471",
    "account_number": "ACC-77210",
    "customer_name": "Meridian Manufacturing",
    "meter_id": "MTR-4402",
    "bill_date": "2026-08-01",
    "total_amount": 482.50,
    "is_open": True,
}

sample_payment_a: PaymentTransactionPayload = {
    "transaction_id": "TXN-11000",
    "account_number": "ACC-77210",
    "customer_name": "Meridian Manufacturing",
    "amount_paid": 482.50,
    "payment_date": "2026-08-10",
    "reference_number": "BILL-4471",
    "payment_channel": "ACH_BANK_TRANSFER",
}

initial_state_a = create_initial_reconciliation_state(sample_payment_a, sample_billing_a, None)

try:
    state_a_step1 = execute_priority_rules_node(initial_state_a)
    print("Example A (Perfect Match) Result:")
    print(f"Priority Rule Matched: {state_a_step1['priority_rule_matched']}")
    print(f"Match Type: {state_a_step1['match_type']}")
    for note in state_a_step1["audit_notes"]:
        print(f" - {note}")
except Exception as err:
    print(f"Priority matching test failed: {err}")
```

OUTPUT

```
Example A (Perfect Match) Result:
Priority Rule Matched: 1
Match Type: 2-Way
- State initialized to OPEN.
- Priority Rule 1 Matched: Account + Bill Number + Amount.
```

This defines the **Smart Matching** use case: when the deterministic rules in the previous cell find no match at all, `smart_matching_node` scores every open bill in a candidate pool on a blended confidence signal — 50% amount closeness, 35% customer-name similarity, 15% date proximity — and suggests the highest-scoring bill above a 0.75 confidence threshold for analyst confirmation. This is the lightweight "AI logic" layer for Day 1; Notebook 2 upgrades it to full vector-embedding retrieval plus a grounded LLM call.

CODE

```
# =====
# Use Case: Smart Matching
# When deterministic Priority Rules 1-6 fail to find a match, Smart
# Matching scores every open bill in the candidate pool on a blended
# confidence signal (amount closeness + name similarity + date proximity)
# and suggests the most probable match for analyst confirmation.
# This is the lightweight "AI logic" layer for Day 1; Notebook 2 upgrades
# it to full vector-embedding + grounded LLM reasoning.
# =====
from datetime import date

def _date_proximity_score(date_str_a: str, date_str_b: str) -> float:
    try:
        d1 = date.fromisoformat(date_str_a)
        d2 = date.fromisoformat(date_str_b)
        delta_days = abs((d1 - d2).days)
        return max(0.0, 1.0 - (delta_days / 30.0))
    except Exception:
        return 0.0

def smart_matching_node(state: Dict[str, Any], candidate_bills: List[Dict[str, Any]]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    payment = state.get("payment_transaction", {})

    if updated_state.get("priority_rule_matched") is not None:
        audit_notes.append("Smart Matching skipped: a deterministic priority rule already matched.")
        updated_state["audit_notes"] = audit_notes
        return updated_state

    scored_candidates = []
    for bill in candidate_bills:
        amount_diff = abs(payment.get("amount_paid", 0.0) - bill.get("total_amount", 0.0))
        amount_score = max(0.0, 1.0 - (amount_diff / max(bill.get("total_amount", 1.0), 1.0)))
        name_score = fuzzy_similarity(payment.get("customer_name", ""),
        bill.get("customer_name", ""))
        date_score = _date_proximity_score(payment.get("payment_date", ""),
        bill.get("bill_date", ""))

        blended_confidence = round((0.5 * amount_score) + (0.35 * name_score) + (0.15 *
date_score), 4)
        scored_candidates.append({
            "bill_id": bill.get("bill_id"),
            "confidence": blended_confidence,
            "amount_score": round(amount_score, 4),
            "name_score": round(name_score, 4),
            "date_score": round(date_score, 4),
        })

    scored_candidates.sort(key=lambda c: c["confidence"], reverse=True)
    updated_state["smart_match_candidates"] = scored_candidates
```

```
if scored_candidates and scored_candidates[0]["confidence"] >= 0.75:
    top = scored_candidates[0]
    updated_state["smart_match_confidence"] = top["confidence"]
    audit_notes.append(
        f"Smart Matching suggests bill {top['bill_id']} as the probable match "
        f"(confidence={top['confidence']:.2f}). Routed for analyst confirmation."
    )
    updated_state["requires_human_review"] = True
else:
    audit_notes.append("Smart Matching found no candidate above the 0.75 confidence
threshold.")

updated_state["audit_notes"] = audit_notes
return updated_state
```

This cell produces no printed output.

Builds a payment with no account number and a slightly misspelled customer name ("Meridian Manufact."), plus a two-bill candidate pool, and runs it through Smart Matching. You should see BILL-4471 suggested as the probable match with a confidence comfortably above the 0.75 threshold, along with the full ranked candidate list showing each component score.

CODE

```
candidate_pool = [
    {"bill_id": "BILL-4471", "account_number": "ACC-77210", "customer_name": "Meridian
Manufacturing",
    "meter_id": "MTR-4402", "bill_date": "2026-08-01", "total_amount": 610.00, "is_open":
True},
    {"bill_id": "BILL-9081", "account_number": "ACC-40021", "customer_name": "Coastal Retail
Group",
    "meter_id": "MTR-1190", "bill_date": "2026-07-28", "total_amount": 220.00, "is_open":
True},
]

sample_payment_smart: PaymentTransactionPayload = {
    "transaction_id": "TXN-9910",
    "account_number": None,
    "customer_name": "Meridian Manufact.",
    "amount_paid": 605.00,
    "payment_date": "2026-08-09",
    "reference_number": None,
    "payment_channel": "MOBILE_WALLET",
}

state_smart = create_initial_reconciliation_state(sample_payment_smart, None, None)
try:
    state_smart = execute_priority_rules_node(state_smart)
    state_smart = smart_matching_node(state_smart, candidate_pool)
    print("Smart Matching Result:")
    for note in state_smart["audit_notes"]:
        print(f" - {note}")
    print("\nRanked Candidates:")
    for c in state_smart["smart_match_candidates"]:
        print(f" {c}")
except Exception as err:
    print(f"Smart matching test failed: {err}")
```

OUTPUT

```
Smart Matching Result:
- State initialized to OPEN.
- No Billing AR record available; no rule can be evaluated.
- Smart Matching suggests bill BILL-4471 as the probable match (confidence=0.90). Routed for
analyst confirmation.

Ranked Candidates:
{'bill_id': 'BILL-4471', 'confidence': 0.9034, 'amount_score': 0.9918, 'name_score': 0.85,
'date_score': 0.7333}
{'bill_id': 'BILL-9081', 'confidence': 0.1453, 'amount_score': 0.0, 'name_score': 0.1579,
'date_score': 0.6}
```

Two node functions. `adjustment_discrepancy_node` compares the billed total against the amount actually paid, computes the variance, and — if the earlier keyword heuristic found an adjustment code — attaches its full category name from `ADJUSTMENT_CATEGORY_NAMES`. `tolerance_auto_write_off_node` then applies the tolerance rule: any variance at or below \$5.00 is automatically written off and the case moves straight to CLOSED; anything larger becomes PARTIAL MATCH, paying the approved portion and opening a dispute item for the rest.

CODE

```
# =====
# Adjustment Code Finalization & Tolerance Buffer
# =====
ADJUSTMENT_CATEGORY_NAMES = {
    "U01": "Meter Reading Dispute",
    "U02": "Service Outage Credit",
    "U03": "Estimated Bill Correction",
    "U04": "Assistance / Discount Adjustment",
    "U05": "Late Fee Waiver",
}

def adjustment_discrepancy_node(state: Dict[str, Any]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    payment = state.get("payment_transaction", {})
    billing = state.get("billing_ar")

    if not billing:
        updated_state["audit_notes"] = audit_notes
        return updated_state

    billed_total = billing.get("total_amount", 0.0)
    paid_amount = payment.get("amount_paid", 0.0)
    variance = round(billed_total - paid_amount, 2)

    code = updated_state.get("adjustment_code")
    category = ADJUSTMENT_CATEGORY_NAMES.get(code) if code else None

    if variance <= 0:
        updated_state["approved_settlement_amount"] = paid_amount
        updated_state["held_variance_amount"] = 0.0
        audit_notes.append("No short payment detected; full billed amount settled.")
    else:
        updated_state["approved_settlement_amount"] = paid_amount
        updated_state["held_variance_amount"] = variance
        updated_state["adjustment_category"] = category
        audit_notes.append(
            f"Short payment of ${variance:,.2f} identified. Mapped to code {code} ({category})."
        )

    updated_state["audit_notes"] = audit_notes
    return updated_state

def tolerance_auto_write_off_node(state: Dict[str, Any], tolerance_limit: float = 5.00) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    held_amount = state.get("held_variance_amount", 0.0)
    billing = state.get("billing_ar") or {}

    if 0.0 < held_amount <= tolerance_limit:
        updated_state["auto_write_off_amount"] = held_amount
        updated_state["approved_settlement_amount"] = billing.get(
```

```

        "total_amount", state.get("payment_transaction", {}).get("amount_paid", 0.0)
    )
    updated_state["held_variance_amount"] = 0.0
    updated_state["current_state"] = "CLOSED"
    audit_notes.append(
        f"Variance of ${held_amount:,.2f} is <= ${tolerance_limit:,.2f} tolerance. Auto-
written off. End State: CLOSED."
    )
    elif held_amount > tolerance_limit:
        updated_state["current_state"] = "PARTIAL MATCH"
        audit_notes.append(
            f"Variance of ${held_amount:,.2f} exceeds ${tolerance_limit:,.2f} tolerance. End
State: PARTIAL MATCH."
        )
    else:
        updated_state["current_state"] = "CLOSED"
        audit_notes.append("Full match with zero variance. End State: CLOSED.")

    updated_state["audit_notes"] = audit_notes
    return updated_state

```

This cell produces no printed output.

`unmatched_exception_node` handles every case where the priority rules found nothing at all: a recognized customer name with no account/bill reference becomes **UAC**, completely blank metadata becomes **UIC**, and anything else ambiguous becomes **QUERY** with `requires_human_review` flipped to True.

`route_reconciliation_workflow` is the router function LangGraph will call after priority matching — if any of Rules 1–6 matched it returns "deduce_adjustment", otherwise "handle_unmatched".

CODE

```
# =====
# Exception Routing (UAC / UIC / QUERY) & Conditional Router
# =====
def unmatched_exception_node(state: Dict[str, Any]) -> Dict[str, Any]:
    updated_state = dict(state)
    audit_notes = list(state.get("audit_notes", []))
    payment = state.get("payment_transaction", {})

    has_customer_name = bool(payment.get("customer_name"))
    has_any_reference = bool(payment.get("account_number") or payment.get("reference_number"))

    if has_customer_name and not has_any_reference and not state.get("billing_ar"):
        updated_state["current_state"] = "UAC"
        audit_notes.append("Customer name recognized but no bill/account reference found. End State: UAC.")
    elif not has_customer_name and not has_any_reference:
        updated_state["current_state"] = "UIC"
        audit_notes.append("Payment metadata is completely blank. End State: UIC.")
    else:
        updated_state["current_state"] = "QUERY"
        updated_state["requires_human_review"] = True
        audit_notes.append("Ambiguous or conflicting metadata. Routed to QUERY for analyst review.")

    updated_state["audit_notes"] = audit_notes
    return updated_state

def route_reconciliation_workflow(state: Dict[str, Any]) -> Literal["deduce_adjustment", "handle_unmatched"]:
    rule_matched = state.get("priority_rule_matched")
    if rule_matched is not None and rule_matched in [1, 2, 3, 4, 5, 6]:
        return "deduce_adjustment"
    return "handle_unmatched"
```

This cell produces no printed output.

Before wiring everything into a compiled LangGraph, this cell exercises the nodes manually in sequence so you can see exactly what each one contributes: Meridian Manufacturing pays \$560.00 against a \$610.00 bill with a dispute note about an estimated meter read. You should see the state land on **PARTIAL MATCH** with adjustment code **U01**, an approved settlement of \$560.00, and a held variance of \$50.00.

CODE

```
# Manual step-by-step run: Example B (Short Payment & Dispute Capture)
sample_billing_b: BillingARPayload = {
    "bill_id": "BILL-4471", "account_number": "ACC-77210", "customer_name": "Meridian
Manufacturing",
    "meter_id": "MTR-4402", "bill_date": "2026-08-01", "total_amount": 610.00, "is_open": True,
}
sample_payment_b: PaymentTransactionPayload = {
    "transaction_id": "TXN-6055", "account_number": "ACC-77210", "customer_name": "Meridian
Manufacturing",
    "amount_paid": 560.00, "payment_date": "2026-08-10", "reference_number": "BILL-4471",
    "payment_channel": "ACH_BANK_TRANSFER",
}
sample_dispute_b: DisputeNotePayload = {
    "dispute_id": "DSP-330", "account_number": "ACC-77210",
    "unstructured_notes": "Meter read appears estimated high in error ($50 credit requested).",
    "source_channel": "EMAIL",
}

state_b = create_initial_reconciliation_state(sample_payment_b, sample_billing_b,
sample_dispute_b)
state_b = data_ingestion_node(state_b)
state_b = remittance_processing_node(state_b)
state_b = execute_priority_rules_node(state_b)

route_b = route_reconciliation_workflow(state_b)
print(f"Dynamic Router Decision for Example B: '{route_b}'")

if route_b == "deduce_adjustment":
    state_b = adjustment_discrepancy_node(state_b)
    state_b = tolerance_auto_write_off_node(state_b)
else:
    state_b = unmatched_exception_node(state_b)

print("\n--- Example B Final Output ---")
print(f"Final State: {state_b['current_state']}")
print(f"Priority Rule Matched: {state_b['priority_rule_matched']}")
print(f"Adjustment Code: {state_b['adjustment_code']} ({state_b['adjustment_category']})")
print(f"Approved Settlement: ${state_b['approved_settlement_amount']:,.2f}")
print(f"Held Variance: ${state_b['held_variance_amount']:,.2f}")
print("Audit Notes:")
for note in state_b["audit_notes"]:
    print(f" - {note}")
```

OUTPUT

```
Dynamic Router Decision for Example B: 'deduce_adjustment'
```

```
--- Example B Final Output ---
```

```
Final State: PARTIAL MATCH
```

```
Priority Rule Matched: 6
```

```
Adjustment Code: U01 (Meter Reading Dispute)
```

```
Approved Settlement: $560.00
```

```
Held Variance: $50.00
```

```
Audit Notes:
```

```
 - State initialized to OPEN.
```

```
 - Data Ingestion validated payment TXN-6055 from channel 'ACH_BANK_TRANSFER'.
```

- Remittance Processing extracted claimed_amount=50.0 keyword_hint_code=U01 from dispute note DSP-330.
- Priority Rule 6 Matched: Account + Bill Number + Amount, reinforced by dispute note (3-Way).
- Short payment of \$50.00 identified. Mapped to code U01 (Meter Reading Dispute).
- Variance of \$50.00 exceeds \$5.00 tolerance. End State: PARTIAL MATCH.

Cell 23 — Manually run Example D (Un-applied Cash — UAC)

Repeats the same manual walkthrough for a payment with no account number at all — \$275.00 from "Sunrise Apartments LLC" with nothing to match it to. Because the customer name is present but there is no bill or account reference and no billing record, this should route to **handle_unmatched** and land on **UAC**.

CODE

```
# Manual step-by-step run: Example D Case 1 (Un-applied Cash - UAC)
sample_payment_d: PaymentTransactionPayload = {
    "transaction_id": "TXN-5000", "account_number": None, "customer_name": "Sunrise Apartments LLC",
    "amount_paid": 275.00, "payment_date": "2026-08-11", "reference_number": None,
    "payment_channel": "ACH_BANK_TRANSFER",
}

state_d = create_initial_reconciliation_state(sample_payment_d, None, None)
state_d = data_ingestion_node(state_d)
state_d = execute_priority_rules_node(state_d)

route_d = route_reconciliation_workflow(state_d)
print(f"Dynamic Router Decision for Example D: '{route_d}'")

if route_d == "deduce_adjustment":
    state_d = adjustment_discrepancy_node(state_d)
    state_d = tolerance_auto_write_off_node(state_d)
else:
    state_d = unmatched_exception_node(state_d)

print("\n--- Example D Final Output ---")
print(f"Final State: {state_d['current_state']}")
print("Audit Notes:")
for note in state_d["audit_notes"]:
    print(f" - {note}")
```

OUTPUT

```
Dynamic Router Decision for Example D: 'handle_unmatched'

--- Example D Final Output ---
Final State: UAC
Audit Notes:
 - State initialized to OPEN.
 - Data Ingestion validated payment TXN-5000 from channel 'ACH_BANK_TRANSFER'.
 - No Billing AR record available; no rule can be evaluated.
 - Customer name recognized but no bill/account reference found. End State: UAC.
```

Records two more telemetry events to `system.log`: one for the Example B state transition (with the approved settlement, held variance, and routing path taken), and one for Example D (noting the routing path and reason). This is the same `JSONStateDeltaLogger.log_state_delta(...)` pattern used throughout the notebook, giving you a consistent, greppable audit trail regardless of whether a state change came from a manual node call or a fully compiled graph invocation.

CODE

```
try:
    telemetry = JSONStateDeltaLogger("system.log")

    telemetry.log_state_delta(
        transaction_id=sample_payment_b["transaction_id"],
        account_number=sample_payment_b["account_number"],
        previous_state="OPEN",
        new_state=state_b["current_state"],
        priority_rule_applied=state_b["priority_rule_matched"],
        adjustment_code=state_b["adjustment_code"],
        delta_payload={
            "approved_settlement": state_b["approved_settlement_amount"],
            "held_variance": state_b["held_variance_amount"],
            "routing_path": route_b,
        },
    )

    telemetry.log_state_delta(
        transaction_id=sample_payment_d["transaction_id"],
        account_number=sample_payment_d["account_number"],
        previous_state="OPEN",
        new_state=state_d["current_state"],
        priority_rule_applied=state_d.get("priority_rule_matched"),
        adjustment_code=None,
        delta_payload={"routing_path": route_d, "reason": "No bill/account reference on
payment"},
    )

    print("Node execution telemetry successfully appended to system.log.")
except Exception as err:
    print(f"Failed to log node execution telemetry: {err}")
```

OUTPUT

```
{
  "timestamp": "2026-08-16T12:56:29.461326+00:00",
  "telemetry_event": "STATE_DELTA_TRANSITION",
  "transaction_id": "TXN-6055",
  "account_number": "ACC-77210",
  "state_transition": {
    "from": "OPEN",
    "to": "PARTIAL MATCH"
  },
  "rule_context": {
    "priority_rule": 6,
    "adjustment_code": "U01"
  },
  "delta_payload": {
    "approved_settlement": 560.0,
    "held_variance": 50.0,
    "routing_path": "deduce_adjustment"
  }
}
{
  "timestamp": "2026-08-16T12:56:29.462486+00:00",
  "telemetry_event": "STATE_DELTA_TRANSITION",
  "transaction_id": "TXN-5000",
  "account_number": null,
  "state_transition": {
    "from": "OPEN",
    "to": "UAC"
  },
  "rule_context": {
    "priority_rule": null,
    "adjustment_code": null
  },
  "delta_payload": {
    "routing_path": "handle_unmatched",
    "reason": "No bill/account reference on payment"
  }
}
Node execution telemetry successfully appended to system.log.
```

Cell 25 — A2: Multi-Agent Orchestration — LangGraph StateGraph

This comment banner marks the A2 module and imports `StateGraph`, `START`, and `END` from `langgraph.graph`, then instantiates `builder = StateGraph(UtilityReconciliationState)`. Passing the TypedDict here tells LangGraph the exact shape of the state object that will flow between nodes, which lets it validate that your node functions are reading and writing the fields you declared.

CODE

```
# =====
# A2: Multi-Agent Orchestration – LangGraph StateGraph
# Wires the nodes above into a compiled, executable graph using a
# Supervisor-style conditional router (the same pattern documented
# in the capstone architecture spec, Section 2).
# =====
from langgraph.graph import StateGraph, START, END

try:
    builder = StateGraph(UtilityReconciliationState)
    print("StateGraph builder initialized with UtilityReconciliationState schema.")
except Exception as err:
    print(f"Failed to initialize StateGraph builder: {err}")
```

OUTPUT

```
StateGraph builder initialized with UtilityReconciliationState schema.
```

Cell 26 — Register the six processing nodes

Adds all six node functions to the builder under short, readable names: `ingest_data`, `process_remittance`, `execute_priority_rules`, `deduce_adjustment`, `tolerance_write_off`, and `handle_unmatched`. Think of this step as registering each function as a labeled stop on the graph.

CODE

```
try:
    builder.add_node("ingest_data", data_ingestion_node)
    builder.add_node("process_remittance", remittance_processing_node)
    builder.add_node("execute_priority_rules", execute_priority_rules_node)
    builder.add_node("deduce_adjustment", adjustment_discrepancy_node)
    builder.add_node("tolerance_write_off", tolerance_auto_write_off_node)
    builder.add_node("handle_unmatched", unmatched_exception_node)
    print("All 6 processing nodes successfully added to StateGraph builder.")
except Exception as err:
    print(f"Failed to add nodes to StateGraph builder: {err}")
```

OUTPUT

```
All 6 processing nodes successfully added to StateGraph builder.
```

Cell 27 — Wire the edges, including the conditional routing edge

This defines how execution actually flows between the six registered nodes: every run starts at `ingest_data`, flows sequentially into `process_remittance` then `execute_priority_rules`, and then `add_conditional_edges(...)` calls `route_reconciliation_workflow` to decide whether to continue to `deduce_adjustment` or jump to `handle_unmatched`. The matched-and-resolved path continues on to `tolerance_write_off` then `END`; the unmatched path ends immediately after classification.

CODE

```
try:
    builder.add_edge(START, "ingest_data")
    builder.add_edge("ingest_data", "process_remittance")
    builder.add_edge("process_remittance", "execute_priority_rules")

    builder.add_conditional_edges(
        "execute_priority_rules",
        route_reconciliation_workflow,
        {
            "deduce_adjustment": "deduce_adjustment",
            "handle_unmatched": "handle_unmatched",
        },
    )

    builder.add_edge("deduce_adjustment", "tolerance_write_off")
    builder.add_edge("tolerance_write_off", END)
    builder.add_edge("handle_unmatched", END)

    print("Graph execution edges and dynamic conditional routing configured successfully.")
except Exception as err:
    print(f"Failed to configure StateGraph edges: {err}")
```

OUTPUT

```
Graph execution edges and dynamic conditional routing configured successfully.
```

Cell 28 — Compile the graph into a runnable app

`app = builder.compile()` turns the node/edge definitions into an executable object. Compilation is also where LangGraph would surface structural errors — for example, a node referenced in an edge that was never registered with `add_node`. A successful print confirms the graph is internally consistent and ready to run.

CODE

```
try:
    app = builder.compile()
    print("StateGraph compiled successfully into an executable runnable application.")
except Exception as err:
    print(f"Failed to compile StateGraph: {err}")
```

OUTPUT

```
StateGraph compiled successfully into an executable runnable application.
```

Cell 29 — Invoke the compiled graph for Example A (perfect match)

Runs the entire compiled graph in one call, `app.invoke(initial_state_a)`, for the Example A payload built back in Cell 15. Unlike the manual step-by-step cells earlier, LangGraph itself now decides the path through the nodes. Expect `current_state` to land on **CLOSED**, Priority Rule 1, match type "2-Way", and the full \$482.50 approved with nothing held back.

CODE

```
try:
    final_state_a = app.invoke(initial_state_a)

    print("\n--- Example A Graph Execution Output ---")
    print(f"Final State: {final_state_a['current_state']}")
    print(f"Priority Rule Matched: {final_state_a['priority_rule_matched']}")
    print(f"Match Type: {final_state_a['match_type']}")
    print(f"Approved Settlement: ${final_state_a['approved_settlement_amount']:, .2f}")
    print("Audit Log Trail:")
    for log in final_state_a["audit_notes"]:
        print(f" - {log}")
except Exception as err:
    print(f"Example A invocation failed: {err}")
```

OUTPUT

```
--- Example A Graph Execution Output ---
Final State: CLOSED
Priority Rule Matched: 1
Match Type: 2-Way
Approved Settlement: $482.50
Audit Log Trail:
- State initialized to OPEN.
- Data Ingestion validated payment TXN-11000 from channel 'ACH_BANK_TRANSFER'.
- Remittance Processing: no dispute/adjustment note attached.
- Priority Rule 1 Matched: Account + Bill Number + Amount.
- No short payment detected; full billed amount settled.
- Full match with zero variance. End State: CLOSED.
```

Cell 30 — Invoke the compiled graph for Example C (tolerance auto write-off)

Builds a new payload — Coastal Retail Group is billed \$150.00 and pays \$148.00, a \$2.00 difference — and runs it through the same compiled `app`. Because \$2.00 is within the tolerance node's \$5.00 tolerance limit, the graph should report `current_state` as **CLOSED**, an auto write-off amount of \$2.00, and the full \$150.00 approved — the small variance is absorbed automatically rather than opening a dispute.

CODE

```
sample_billing_c: BillingARPayload = {
    "bill_id": "BILL-2290", "account_number": "ACC-88510", "customer_name": "Coastal Retail
Group",
    "meter_id": "MTR-1190", "bill_date": "2026-08-02", "total_amount": 150.00, "is_open": True,
}
sample_payment_c: PaymentTransactionPayload = {
    "transaction_id": "TXN-9002", "account_number": "ACC-88510", "customer_name": "Coastal
Retail Group",
    "amount_paid": 148.00, "payment_date": "2026-08-09", "reference_number": "BILL-2290",
    "payment_channel": "ONLINE_BILLPAY",
}
initial_state_c = create_initial_reconciliation_state(sample_payment_c, sample_billing_c, None)

try:
    final_state_c = app.invoke(initial_state_c)

    print("\n--- Example C Graph Execution Output (Tolerance) ---")
    print(f"Final State: {final_state_c['current_state']}")
    print(f"Priority Rule Matched: {final_state_c['priority_rule_matched']}")
    print(f"Auto Write-Off Amount: ${final_state_c['auto_write_off_amount']:,.2f}")
    print(f"Approved Settlement: ${final_state_c['approved_settlement_amount']:,.2f}")
    print("Audit Log Trail:")
    for log in final_state_c["audit_notes"]:
        print(f" - {log}")
except Exception as err:
    print(f"Example C invocation failed: {err}")
```

OUTPUT

```
--- Example C Graph Execution Output (Tolerance) ---
Final State: CLOSED
Priority Rule Matched: 3
Auto Write-Off Amount: $2.00
Approved Settlement: $150.00
Audit Log Trail:
- State initialized to OPEN.
- Data Ingestion validated payment TXN-9002 from channel 'ONLINE_BILLPAY'.
- Remittance Processing: no dispute/adjustment note attached.
- Priority Rule 3 Matched: Account + Bill Number + Bill Date + Amount.
- Short payment of $2.00 identified. Mapped to code None (None).
- Variance of $2.00 is <= $5.00 tolerance. Auto-written off. End State: CLOSED.
```

Logs two more telemetry events, tagged `"execution_type": "Compiled_StateGraph_Invoke"` to distinguish them from the manual node-by-node runs logged earlier — one capturing Example A's clean \$482.50 settlement, and one capturing Example C's auto write-off.

CODE

```
try:
    telemetry = JSONStateDeltaLogger("system.log")

    telemetry.log_state_delta(
        transaction_id=sample_payment_a["transaction_id"],
        account_number=sample_payment_a["account_number"],
        previous_state="OPEN",
        new_state=final_state_a["current_state"],
        priority_rule_applied=final_state_a["priority_rule_matched"],
        adjustment_code=None,
        delta_payload={
            "approved_settlement": final_state_a["approved_settlement_amount"],
            "execution_type": "Compiled_StateGraph_Invoke",
        },
    )

    telemetry.log_state_delta(
        transaction_id=sample_payment_c["transaction_id"],
        account_number=sample_payment_c["account_number"],
        previous_state="OPEN",
        new_state=final_state_c["current_state"],
        priority_rule_applied=final_state_c["priority_rule_matched"],
        adjustment_code=final_state_c.get("adjustment_code"),
        delta_payload={
            "auto_write_off": final_state_c["auto_write_off_amount"],
            "execution_type": "Compiled_StateGraph_Invoke",
        },
    )

    print("\nCompiled graph execution results logged to system.log successfully.")
except Exception as err:
    print(f"Failed to record state graph execution telemetry: {err}")
```

OUTPUT

```
{"timestamp": "2026-08-16T12:56:30.977233+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "TXN-11000", "account_number": "ACC-77210", "state_transition": {"from":
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 1, "adjustment_code": null},
"delta_payload": {"approved_settlement": 482.5, "execution_type":
"Compiled_StateGraph_Invoke"}}
{"timestamp": "2026-08-16T12:56:30.978310+00:00", "telemetry_event": "STATE_DELTA_TRANSITION",
"transaction_id": "TXN-9002", "account_number": "ACC-88510", "state_transition": {"from":
"OPEN", "to": "CLOSED"}, "rule_context": {"priority_rule": 3, "adjustment_code": null},
"delta_payload": {"auto_write_off": 2.0, "execution_type": "Compiled_StateGraph_Invoke"}}

Compiled graph execution results logged to system.log successfully.
```

Cell 32 — Print the Day 1 capabilities summary

A final wrap-up cell that prints a short checklist of everything this notebook built: the A1 deterministic Planner-Executor nodes, the A2 compiled LangGraph StateGraph, all four Day 1 use cases (Data Ingestion, Remittance Processing, Payment Matching, Smart Matching), and the six supported end states. It also previews what Notebook 2 adds next.

CODE

```
print("=" * 70)
print("DAY 1 CAPABILITIES BUILT")
print("=" * 70)
print("- A1 Agent Architecture: deterministic Planner-Executor nodes")
print("- A2 Multi-Agent Orchestration: compiled LangGraph StateGraph")
print("- Data Ingestion, Remittance Processing, Payment Matching,")
print("  and Smart Matching use cases all implemented and tested")
print("- 6 End States supported: OPEN, PARTIAL MATCH, CLOSED, UAC, UIC, QUERY")
print("\nNext: Notebook 2 adds MCP/A2A integration, vector RAG grounding,")
print("a voice-intake lab, and an agent evaluation harness (Day 2 / Track B).")
```

OUTPUT

```
=====
DAY 1 CAPABILITIES BUILT
=====
- A1 Agent Architecture: deterministic Planner-Executor nodes
- A2 Multi-Agent Orchestration: compiled LangGraph StateGraph
- Data Ingestion, Remittance Processing, Payment Matching,
  and Smart Matching use cases all implemented and tested
- 6 End States supported: OPEN, PARTIAL MATCH, CLOSED, UAC, UIC, QUERY

Next: Notebook 2 adds MCP/A2A integration, vector RAG grounding,
a voice-intake lab, and an agent evaluation harness (Day 2 / Track B).
```

End of Lab Guide — Automated Utility & Telecom Billing Reconciliation Capstone, Day 1